UNIVERSIDADE PAULISTA – UNIP
CIÊNCIA DA COMPUTAÇÃO
CAMPUS CHÁCARA SANTO ANTÔNIO II

CAIO HENRIQUE NUNES DOS SANTOS – RA: G79ADD7
CARLOS EDUARDO BISPO SOUZA – RA: G7280A2

DESENVOLVIMENTO DE UMA APLICAÇÃO UTILIZANDO WEB SERVICES:

EcoGestão – sistema de gestão ambiental baseado na ISO 14001

SÃO PAULO
2026

CAIO HENRIQUE NUNES DOS SANTOS – RA: G79ADD7
CARLOS EDUARDO BISPO SOUZA – RA: G7280A2

DESENVOLVIMENTO DE UMA APLICAÇÃO UTILIZANDO WEB SERVICES:

EcoGestão – sistema de gestão ambiental baseado na ISO 14001

Atividades Práticas Supervisionadas (APS) apresentadas ao curso de Ciência da Computação da Universidade Paulista – UNIP, como requisito parcial para aprovação no 7º e 8º semestres. Tema: desenvolvimento de uma aplicação utilizando web services.

Orientador: Prof. Carlos Baltazar

SÃO PAULO

2026

RESUMO

Este trabalho apresenta o desenvolvimento do EcoGestão, uma aplicação baseada em web services voltada para a gestão ambiental de uma organização, de acordo com os princípios da norma ISO 14001. O objetivo foi construir um serviço web no estilo REST que permita registrar e acompanhar a geração e a destinação de resíduos, o consumo mensal de água e de energia e o cadastro de fornecedores e parceiros, que são partes interessadas (stakeholders) da empresa, verificando se eles possuem certificação ambiental. Também foi desenvolvida uma aplicação cliente, executada no navegador, que consome esse serviço por meio de requisições HTTP com dados no formato JSON. O serviço foi implementado em Python com o microframework Flask e banco de dados SQLite, e o cliente foi feito com HTML, CSS e JavaScript. O sistema conta com validação dos dados, tratamento de erros com códigos de status HTTP adequados, filtros, um painel com indicadores ambientais, como o percentual de resíduos reaproveitados e a variação do consumo de recursos, além da exportação de relatório em formato CSV e de testes automatizados. O texto apresenta ainda os fundamentos dos web services, o plano de desenvolvimento, o projeto do programa e o código-fonte, e discute a contribuição do trabalho para a formação do grupo e a interdisciplinaridade envolvida. Concluiu-se que os web services facilitam a integração entre sistemas e que a tecnologia pode apoiar de forma simples o monitoramento ambiental exigido por um sistema de gestão ambiental.

Palavras-chave: Web services; REST; Gestão ambiental; ISO 14001; Python.

ABSTRACT

This work presents the development of EcoGestão, an application based on web services for the environmental management of an organization, following the principles of the ISO 14001 standard. The goal was to build a REST web service that allows the registration and monitoring of waste generation and disposal, the monthly consumption of water and energy, and the registration of suppliers and partners, who are stakeholders of the company, checking whether they hold an environmental certification. A client application that runs in the web browser was also developed, consuming the service through HTTP requests with data in JSON format. The service was implemented in Python with the Flask microframework and an SQLite database, and the client was built with HTML, CSS and JavaScript. The system provides data validation, error handling with proper HTTP status codes, filters, a dashboard with environmental indicators, such as the percentage of reused waste and the variation in resource consumption, in addition to a report export in CSV format and automated tests. The text also presents the fundamentals of web services, the development plan, the program design and the source code, and discusses the contribution of this work to the education of the group and the interdisciplinarity involved. It was concluded that web services make integration between systems easier and that technology can support, in a simple way, the environmental monitoring required by an environmental management system.

Keywords: Web services; REST; Environmental management; ISO 14001; Python.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura do sistema EcoGestão.....25
Figura 2 – Modelo do banco de dados.....26
Figura 3 – Painel de indicadores ambientais.....40
Figura 4 – Tela de controle de resíduos.....40
Figura 5 – Tela de consumo de água e energia.....41
Figura 6 – Tela de fornecedores.....41
Figura 7 – Resposta em JSON do web service.....42
Figura 8 – Resposta de erro 404 do web service.....42
Figura 9 – Ficha de APS – Caio Henrique Nunes dos Santos.....47
Figura 10 – Ficha de APS – Carlos Eduardo Bispo Souza.....48

Quadro 1 – Comparação entre web services SOAP e REST.....18
Quadro 2 – Relação entre os módulos do sistema e a ISO 14001.....20
Quadro 3 – Requisitos funcionais.....20
Quadro 4 – Requisitos não funcionais.....21
Quadro 5 – Ferramentas e tecnologias utilizadas.....22
Quadro 6 – Cronograma de desenvolvimento.....24
Quadro 7 – Endpoints do web service.....28
Quadro 8 – Códigos HTTP utilizados pelo web service.....28

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	Application Programming Interface (Interface de Programação de Aplicações)
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
PDCA	Plan, Do, Check, Act
REST	Representational State Transfer
SGA	Sistema de Gestão Ambiental
SOA	Service-Oriented Architecture (Arquitetura Orientada a Serviços)
SOAP	Simple Object Access Protocol
SQL	Structured Query Language
UDDI	Universal Description, Discovery and Integration
URI	Uniform Resource Identifier
W3C	World Wide Web Consortium
WSDL	Web Services Description Language
XML	eXtensible Markup Language

SUMÁRIO

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO....8
1.1 Objetivo geral....8
1.2 Objetivos específicos....8
1.3 Motivação....8
2 INTRODUÇÃO....10
3 FUNDAMENTOS E PRINCIPAIS ELEMENTOS DOS WEB SERVICES....13
3.1 Conceito de web service....13
3.2 Arquitetura orientada a serviços....14
3.3 Protocolo HTTP....14
3.4 Formatos de dados: XML e JSON....15
3.5 Web services SOAP....16
3.6 Web services REST....16
3.7 Comparação entre SOAP e REST....17
3.8 Segurança e outros cuidados....18
4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO....19
4.1 Escolha do tema e relação com a ISO 14001....19
4.2 Levantamento de requisitos....20
4.2.1 Regras de negócio....21
4.3 Ferramentas e tecnologias utilizadas....21
4.4 Metodologia de desenvolvimento....23
4.5 Cronograma....23
4.6 Divisão de tarefas....24
4.7 Recursos necessários....24
5 PROJETO (ESTRUTURA E MÓDULOS) DO PROGRAMA....25
5.1 Arquitetura do sistema....25
5.2 Estrutura de pastas....25
5.3 Modelo de dados....26
5.4 Módulos do sistema....26
5.4.1 Módulo de resíduos....26
5.4.2 Módulo de consumo de água e energia....27
5.4.3 Módulo de fornecedores....27
5.4.4 Módulo de relatórios....27

5.5 Endpoints do web service.....27
5.6 Tratamento de erros.....28
5.7 Testes.....28
6 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA.....30
6.1 Arquivo servidor/banco.py.....30
6.2 Arquivo servidor/app.py.....31
6.3 Arquivo cliente/app.js.....36
7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO.....39
7.1 Como executar o sistema.....39
7.2 Telas do sistema.....39
8 CONSIDERAÇÕES FINAIS.....43
8.1 Elementos utilizados no desenvolvimento.....43
8.2 Efeito do trabalho na formação.....44
8.3 Interdisciplinaridade.....44
REFERÊNCIAS.....45
APÊNDICE A – FICHAS DE ATIVIDADES PRÁTICAS SUPERVISIONADAS.....47

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO

1.1 Objetivo geral

O objetivo deste trabalho é desenvolver uma aplicação baseada em web services que auxilie uma organização a realizar a sua gestão ambiental, registrando e acompanhando ações que ajudam a diminuir os impactos ambientais causados pelas suas atividades, de acordo com os princípios da norma ISO 14001. A aplicação é formada por duas partes: um web service, que disponibiliza as funcionalidades do sistema pela rede, e uma aplicação cliente, que consome esse serviço e apresenta as informações para o usuário.

1.2 Objetivos específicos

Para atingir o objetivo geral, o grupo definiu os seguintes objetivos específicos:

a) estudar os conceitos e os principais elementos dos web services, como HTTP, XML, JSON, SOAP e REST;

b) implementar um web service REST para o controle de resíduos, do consumo de água e energia e dos fornecedores da organização;

c) implementar uma aplicação cliente que consuma o web service;

d) gerar indicadores e relatórios que ajudem no acompanhamento do desempenho ambiental;

e) aplicar tratamento de erros e validação dos dados recebidos pelo serviço;

f) documentar todo o processo de desenvolvimento seguindo as normas da ABNT.

1.3 Motivação

A preocupação com o meio ambiente deixou de ser apenas um assunto de órgãos ambientais e passou a fazer parte do dia a dia das empresas. Clientes, governo, comunidade e investidores cobram cada vez mais que as organizações mostrem o que estão fazendo para reduzir o desperdício e a poluição. Muitas pequenas e médias empresas, porém, ainda controlam essas informações em

planilhas soltas ou em papel, o que dificulta saber, por exemplo, quanto lixo foi gerado no mês ou se o consumo de energia está aumentando.

Ao mesmo tempo, os web services são hoje a forma mais comum de fazer sistemas diferentes conversarem entre si. Praticamente todo aplicativo de celular, site ou sistema empresarial moderno consome ou oferece algum serviço web. Por isso, o grupo viu neste trabalho uma boa oportunidade de unir um tema importante para a sociedade, que é a gestão ambiental, com uma tecnologia muito usada no mercado de trabalho da área de computação. Além disso, um sistema construído como serviço pode ser usado futuramente por outros clientes, como um aplicativo de celular ou um sistema de outra empresa, sem precisar refazer as regras de negócio.

2 INTRODUÇÃO

A forma como os sistemas de computador são construídos mudou bastante nas últimas décadas. No começo, a maioria dos programas era feita para rodar em um único computador, com todas as suas partes juntas. Com a popularização da internet, as empresas passaram a precisar que seus sistemas trocassem informações com sistemas de clientes, fornecedores, bancos e governo, que muitas vezes foram escritos em linguagens diferentes e rodam em sistemas operacionais diferentes. Os web services surgiram justamente para resolver esse problema, oferecendo uma maneira padronizada de um programa disponibilizar funcionalidades para outros programas através da rede, usando protocolos abertos como o HTTP (W3C, 2004).

Hoje existem dois estilos principais de web services. O primeiro é baseado no protocolo SOAP, que utiliza mensagens em XML e contratos descritos em WSDL, e foi muito usado em sistemas corporativos. O segundo é o estilo REST, proposto por Fielding (2000) em sua tese de doutorado, que aproveita os próprios métodos do HTTP para manipular recursos identificados por endereços (URIs) e que normalmente usa o formato JSON para representar os dados. O estilo REST se tornou o mais utilizado em aplicações web e móveis por ser mais simples e mais leve, e por isso foi o escolhido para este trabalho.

O tema proposto pela APS une essa tecnologia à gestão ambiental. A ISO 14000 é uma família de normas internacionais que oferece orientações para que as organizações controlem os impactos das suas atividades no meio ambiente. A principal delas é a ISO 14001, que define os requisitos de um Sistema de Gestão Ambiental (SGA) e que pode ser usada para certificação (ABNT, 2015). A norma segue o ciclo PDCA (planejar, fazer, verificar e agir), o que significa que a empresa precisa planejar as suas ações ambientais, executá-las, medir os resultados e corrigir o que não está bom, de forma contínua. Para medir resultados é preciso ter dados confiáveis, e é nesse ponto que um sistema de informação pode ajudar.

Outro ponto importante da ISO 14001 é a atenção às partes interessadas, ou stakeholders, que são as pessoas e organizações que afetam ou são afetadas pela empresa, como funcionários, fornecedores, clientes, comunidade e órgãos públicos. A norma pede que a organização entenda as necessidades dessas partes e controle os processos que são feitos por terceiros, como a coleta e o transporte de resíduos

(ABNT, 2015). No Brasil, a Política Nacional de Resíduos Sólidos também determina que os geradores de resíduos sejam responsáveis pela sua destinação correta, dando prioridade para a não geração, a redução, a reutilização e a reciclagem, e deixando o aterro apenas para os rejeitos (Brasil, 2010).

Na prática, porém, muitas organizações têm dificuldade em acompanhar esses indicadores. As informações sobre resíduos ficam nos comprovantes das empresas de coleta, o consumo de água e energia fica nas contas mensais e os dados dos fornecedores ficam espalhados em contratos e planilhas. Quando chega o momento de uma auditoria ou de avaliar se as metas ambientais foram cumpridas, é preciso juntar tudo manualmente, o que toma tempo e aumenta a chance de erros. Um sistema simples, que centralize esses dados e calcule automaticamente os principais indicadores, já representa um grande avanço para esse tipo de empresa.

Também é importante que esse sistema consiga conversar com outros sistemas. Uma empresa pode querer, por exemplo, que o sistema da empresa de coleta envie automaticamente os dados de cada retirada de resíduos, ou que o consumo de energia seja lido diretamente de um medidor inteligente. Isso só é possível de forma prática se as funcionalidades estiverem disponíveis como serviços na rede, o que reforça a escolha dos web services como base da aplicação.

Pensando nisso, o grupo desenvolveu o EcoGestão, um sistema que permite que a organização registre os resíduos que gera, informando o tipo, a quantidade, o destino e a empresa responsável pela coleta, registre o consumo mensal de água e energia elétrica e mantenha um cadastro de fornecedores indicando quais possuem certificação ISO 14001. A partir desses dados, o sistema calcula indicadores como o total de resíduos gerados, o percentual de resíduos que foram reaproveitados (reciclagem, compostagem ou reuso) e a variação do consumo de recursos, que servem como apoio para a etapa de verificação do ciclo PDCA.

Todas essas funcionalidades foram construídas como um web service REST, desenvolvido na linguagem Python com o microframework Flask e banco de dados SQLite. Para mostrar o uso do serviço, foi feita uma aplicação cliente em HTML, CSS e JavaScript, que roda no navegador e se comunica com o serviço apenas por meio de requisições HTTP. Dessa forma, fica claro o papel de cada parte: o serviço guarda os dados e as regras, e o cliente apenas apresenta as informações e envia o que o usuário digitou.

Este trabalho está organizado da seguinte forma: a seção 1 apresentou o objetivo e a motivação; a seção 3 apresenta os fundamentos e os principais elementos dos web services; a seção 4 traz o plano de desenvolvimento, com os requisitos, as ferramentas e o cronograma; a seção 5 descreve o projeto, com a arquitetura, o banco de dados e os módulos do programa; a seção 6 contém o relatório com as linhas de código; a seção 7 mostra o programa em funcionamento; e a seção 8 traz as considerações finais, discutindo a contribuição do trabalho para a formação do grupo e a interdisciplinaridade envolvida. O código-fonte completo também está disponível no repositório do grupo no GitHub (https://github.com/Caiohere/APS---8-Semestre).

3 FUNDAMENTOS E PRINCIPAIS ELEMENTOS DOS WEB SERVICES

Nesta seção são apresentados os conceitos gerais que foram estudados pelo grupo para o desenvolvimento do trabalho, começando pela definição de web service e passando pelos protocolos, formatos de dados e estilos de arquitetura mais utilizados.

3.1 Conceito de web service

De forma simples, um web service é um programa que fica disponível em um servidor e que pode ser chamado por outros programas através da rede, normalmente a internet. A diferença para um site comum é que o site é feito para ser lido por pessoas, enquanto o web service é feito para ser usado por outros sistemas. O W3C, que é o consórcio responsável pelos padrões da web, define web service da seguinte maneira:

> Um web service é um sistema de software projetado para dar suporte à interação interoperável entre máquinas através de uma rede. Ele possui uma interface descrita em um formato que pode ser processado por máquinas (especificamente WSDL). Outros sistemas interagem com o web service da maneira prescrita pela sua descrição usando mensagens SOAP, normalmente transportadas por HTTP com uma serialização XML em conjunto com outros padrões relacionados à web (W3C, 2004, tradução nossa).

Essa definição é a mais antiga e está ligada ao modelo SOAP. Com o tempo o termo passou a ser usado de forma mais ampla, incluindo também os serviços REST, que não usam WSDL nem SOAP. O ponto principal, que vale para os dois casos, é a interoperabilidade: como a comunicação é feita com padrões abertos, um serviço escrito em Python pode ser consumido por um cliente escrito em Java, C#, JavaScript ou qualquer outra linguagem, rodando em qualquer sistema operacional.

Entre as vantagens dos web services estão a reutilização das funcionalidades, já que um mesmo serviço pode atender vários clientes diferentes, a integração entre sistemas de empresas diferentes, o baixo acoplamento, pois o cliente só precisa conhecer a interface do serviço e não como ele foi implementado, e a facilidade de passar por firewalls, porque normalmente usam as mesmas portas da web. Como desvantagens, podem ser citados o desempenho menor em

comparação com chamadas locais, a dependência da rede e a necessidade de cuidar da segurança das informações que trafegam.

3.2 Arquitetura orientada a serviços

Os web services estão ligados ao conceito de Arquitetura Orientada a Serviços (SOA). Segundo Erl (2009), na SOA a lógica de negócio de uma organização é dividida em serviços independentes, com contratos bem definidos, que podem ser combinados para formar soluções maiores. Cada serviço deve ser reutilizável, autônomo, com baixo acoplamento e deve esconder os detalhes da sua implementação.

No modelo clássico da SOA existem três papéis: o provedor do serviço, que é quem implementa e disponibiliza o serviço; o consumidor, que é o cliente que utiliza o serviço; e o registro, que é um catálogo onde os serviços são publicados para que os consumidores possam encontrá-los. Neste trabalho, o servidor Flask é o provedor, a página web é o consumidor e, como o sistema é pequeno, o papel de registro é cumprido pela própria documentação dos endpoints apresentada na seção 5.

3.3 Protocolo HTTP

O HTTP (Hypertext Transfer Protocol) é o protocolo de comunicação da web e é a base da maioria dos web services. Ele funciona no modelo de requisição e resposta: o cliente envia uma requisição para um endereço (URI) do servidor e o servidor devolve uma resposta. Cada requisição possui um método, um caminho, cabeçalhos e, às vezes, um corpo com dados. A resposta possui um código de status, cabeçalhos e um corpo (Fielding; Nottingham; Reschke, 2022).

Os métodos indicam a ação que o cliente quer realizar. Os mais usados em web services são:

a) GET: busca a representação de um recurso, sem alterar nada no servidor;

b) POST: envia dados para o servidor, geralmente para criar um novo recurso;

c) PUT: substitui um recurso existente pelos dados enviados;

d) DELETE: remove um recurso.

Os códigos de status informam o resultado da requisição e são divididos em faixas: os códigos 2xx indicam sucesso (por exemplo, 200 OK e 201 Created), os 3xx indicam redirecionamento, os 4xx indicam erro causado pelo cliente (como 400 Bad Request, 404 Not Found e 409 Conflict) e os 5xx indicam erro no servidor (como 500 Internal Server Error). O uso correto desses códigos é importante porque permite que o cliente saiba o que aconteceu sem precisar interpretar o texto da resposta.

Outra característica importante do HTTP é que ele é um protocolo sem estado (stateless), ou seja, o servidor não guarda informações sobre as requisições anteriores do cliente. Cada requisição precisa carregar tudo o que é necessário para ser atendida. Isso deixa o servidor mais simples e facilita atender muitos clientes ao mesmo tempo.

3.4 Formatos de dados: XML e JSON

Para que dois sistemas troquem informações, eles precisam concordar com um formato para os dados. Os dois formatos mais usados em web services são o XML e o JSON.

O XML (eXtensible Markup Language) é uma linguagem de marcação que organiza os dados em elementos delimitados por tags, parecido com o HTML, mas permitindo que o próprio usuário crie as tags. Ele é muito flexível, permite validar a estrutura dos documentos com esquemas (XSD) e é a base do SOAP e do WSDL. Por outro lado, os documentos XML costumam ser grandes e mais trabalhosos de ler e processar.

O JSON (JavaScript Object Notation) é um formato de texto leve, baseado na forma como os objetos são escritos em JavaScript. Ele representa os dados com objetos (pares de nome e valor entre chaves), listas (entre colchetes), textos, números, valores lógicos e nulo (Bray, 2017). Por ser mais compacto e fácil de ler, e por ser entendido de forma nativa pelos navegadores, o JSON se tornou o formato padrão das APIs REST. O exemplo abaixo mostra como um resíduo cadastrado no sistema deste trabalho é representado em JSON:

```
{
  "id": 1,
  "tipo": "Papel",
  "quantidade_kg": 120.5,
  "destino": "Reciclagem",
```

```
  "data": "2026-09-05",
  "fornecedor": "Recicla Mais Ltda"
}
```

3.5 Web services SOAP

O SOAP (originalmente Simple Object Access Protocol) é um protocolo de troca de mensagens baseado em XML, padronizado pelo W3C. Uma mensagem SOAP é formada por um envelope, que possui um cabeçalho opcional (header), usado para informações como segurança e transações, e um corpo (body), que contém os dados da chamada ou da resposta. Quando acontece um erro, o serviço devolve um elemento especial chamado fault. O SOAP é independente do protocolo de transporte, mas na prática quase sempre é usado sobre HTTP (W3C, 2004).

Junto com o SOAP são usados outros dois padrões. O WSDL (Web Services Description Language) é um documento XML que descreve o serviço, ou seja, quais operações ele oferece, quais parâmetros cada uma recebe, o que devolve e em qual endereço o serviço está. É uma espécie de contrato entre o provedor e o consumidor, e muitas ferramentas conseguem gerar automaticamente o código do cliente a partir dele. Já o UDDI (Universal Description, Discovery and Integration) foi criado para ser um registro onde as empresas publicariam seus serviços para que outras pudessem encontrá-los, cumprindo o papel de registro da SOA. Na prática o UDDI acabou sendo pouco usado.

Os serviços SOAP ainda são bastante encontrados em sistemas bancários, governamentais e corporativos, pois contam com padrões complementares para segurança (WS-Security), transações e mensagens confiáveis. Um exemplo conhecido no Brasil é a Nota Fiscal Eletrônica, cujos serviços de autorização utilizam SOAP.

3.6 Web services REST

O REST (Representational State Transfer) não é um protocolo, mas um estilo de arquitetura, descrito por Roy Fielding na sua tese de doutorado no ano 2000. Fielding (2000) definiu o REST a partir de um conjunto de restrições que, quando seguidas, dão ao sistema características como desempenho, escalabilidade e simplicidade. As principais restrições são:

a) cliente-servidor: a interface com o usuário fica separada do armazenamento dos dados, o que permite que as duas partes evoluam de forma independente;

b) sem estado (stateless): cada requisição deve conter todas as informações necessárias, e o servidor não guarda a sessão do cliente;

c) cache: as respostas devem indicar se podem ou não ser guardadas em cache, para diminuir o número de requisições;

d) interface uniforme: todos os recursos são acessados da mesma forma, usando URIs para identificá-los e os métodos padrão do HTTP para manipulá-los;

e) sistema em camadas: o cliente não precisa saber se está falando direto com o servidor final ou com um intermediário, como um balanceador de carga;

f) código sob demanda (opcional): o servidor pode enviar código para ser executado no cliente.

O conceito central do REST é o de recurso. Um recurso é qualquer informação que possa ter um nome, como um resíduo, um fornecedor ou um relatório. Cada recurso é identificado por uma URI, por exemplo /api/residuos/5, e o cliente manipula esse recurso por meio de representações, geralmente em JSON. Assim, para buscar o resíduo 5 o cliente faz um GET nessa URI, para alterá-lo faz um PUT e para excluí-lo faz um DELETE. Richardson e Ruby (2007) chamam essa forma de projetar de arquitetura orientada a recursos e destacam que ela aproveita tudo o que o HTTP já oferece, em vez de criar um protocolo novo por cima dele.

Quando uma API segue essas restrições, costuma-se dizer que ela é RESTful. Na prática, muitas APIs seguem apenas parte das restrições, mas mesmo assim são chamadas de REST. A API deste trabalho segue as principais: separação entre cliente e servidor, ausência de estado, uso de URIs para identificar recursos, uso dos métodos HTTP e respostas em JSON com códigos de status adequados.

3.7 Comparação entre SOAP e REST

O Quadro 1 resume as principais diferenças entre os dois estilos de web services estudados.

Quadro 1 – Comparação entre web services SOAP e REST

Característica	**SOAP**	**REST**
Tipo	Protocolo	Estilo de arquitetura
Formato das mensagens	Somente XML	JSON, XML, texto e outros (mais comum JSON)
Descrição do serviço	WSDL (contrato formal)	Documentação da API (ex.: OpenAPI)
Transporte	HTTP, SMTP e outros	HTTP
Uso dos métodos HTTP	Normalmente só POST	GET, POST, PUT, DELETE e outros
Tamanho das mensagens	Maior (envelope XML)	Menor
Segurança	WS-Security, padrões próprios	HTTPS, tokens (ex.: OAuth, JWT)
Uso mais comum	Sistemas corporativos e governo	Aplicações web e móveis

Fonte: Autores, 2026.

A escolha do REST para este trabalho foi feita porque o sistema é pequeno, será consumido por uma página web e não precisa dos recursos avançados de transações e segurança do SOAP. Além disso, o JSON é lido diretamente pelo JavaScript do navegador, o que deixou o cliente bem mais simples.

3.8 Segurança e outros cuidados

Como os web services ficam expostos na rede, é preciso ter cuidado com a segurança. Os principais pontos são: usar HTTPS para criptografar a comunicação, exigir autenticação dos clientes (por exemplo com tokens), validar todos os dados recebidos, para evitar erros e ataques como a injeção de SQL, e não mostrar detalhes internos do servidor nas mensagens de erro.

Outro ponto que apareceu durante o desenvolvimento foi o CORS (Cross-Origin Resource Sharing). Por segurança, os navegadores bloqueiam que uma página carregada de um endereço faça requisições para outro endereço, a não ser que o servidor permita isso por meio de cabeçalhos HTTP específicos (Mozilla, 2024). Como o cliente pode ser aberto de outro local, o serviço deste trabalho envia o cabeçalho Access-Control-Allow-Origin. No projeto também foram usadas consultas SQL parametrizadas, que evitam a injeção de SQL, e todas as entradas são validadas antes de serem gravadas. Por ser um trabalho acadêmico, não foi implementada autenticação, o que fica como sugestão de melhoria.

4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO

Nesta seção são apresentados o planejamento do sistema, a relação do tema com a ISO 14001, os requisitos levantados, as ferramentas escolhidas e a forma como o grupo organizou o trabalho ao longo do semestre.

4.1 Escolha do tema e relação com a ISO 14001

A proposta da APS pede que os serviços implementados estejam ligados à gestão ambiental de uma organização, com ações que minimizem os impactos ambientais e melhorem os procedimentos entre as partes interessadas. O grupo começou estudando a norma ISO 14001 para entender o que uma empresa precisa controlar para ter um bom sistema de gestão ambiental.

Segundo Barbieri (2016), a gestão ambiental empresarial envolve as atividades administrativas e operacionais que a empresa realiza para diminuir ou eliminar os danos ambientais causados pelas suas atividades. A ISO 14001 organiza essas atividades no ciclo PDCA. Na etapa de planejamento a empresa identifica os seus aspectos ambientais, que são os elementos das suas atividades que interagem com o meio ambiente, como a geração de resíduos e o consumo de água e energia. Na etapa de execução ela coloca em prática os controles. Na etapa de verificação ela monitora e mede o seu desempenho ambiental. Por fim, na etapa de ação ela corrige os problemas e busca a melhoria contínua (ABNT, 2015).

Analisando os aspectos ambientais mais comuns em empresas de qualquer porte, o grupo escolheu três frentes para o sistema:

a) resíduos sólidos: registrar o que é gerado, a quantidade e para onde vai, incentivando a reciclagem e a compostagem em vez do aterro, como determina a Política Nacional de Resíduos Sólidos (Brasil, 2010);

b) consumo de recursos naturais: registrar mensalmente o consumo de água e de energia elétrica para acompanhar se as ações de economia estão dando resultado;

c) partes interessadas: manter o cadastro dos fornecedores e prestadores de serviço, como empresas de coleta e transporte de resíduos, indicando quais possuem certificação ISO 14001, já que a norma pede o controle dos processos feitos por terceiros.

Com esses dados registrados, o sistema consegue gerar indicadores de desempenho ambiental, que são exatamente o que a etapa de verificação do PDCA exige. O Quadro 2 mostra a ligação entre os módulos do sistema e os requisitos da norma.

Quadro 2 – Relação entre os módulos do sistema e a ISO 14001

Módulo do sistema	**Etapa do PDCA**	**Relação com a ISO 14001**
Resíduos	Fazer / Verificar	Controle operacional dos aspectos ambientais e registro da destinação
Água e energia	Verificar	Monitoramento e medição do uso de recursos
Fornecedores	Planejar / Fazer	Partes interessadas e controle de processos terceirizados
Painel e relatórios	Verificar / Agir	Avaliação do desempenho ambiental e melhoria contínua

Fonte: Autores, 2026.

4.2 Levantamento de requisitos

Depois de definido o tema, o grupo fez o levantamento dos requisitos do sistema. Sommerville (2011) explica que os requisitos funcionais descrevem o que o sistema deve fazer, enquanto os requisitos não funcionais descrevem restrições e qualidades, como desempenho, usabilidade e tecnologia. Os requisitos foram levantados em reuniões do grupo, imaginando como seria o uso do sistema pelo responsável pela área ambiental de uma empresa. O Quadro 3 apresenta os requisitos funcionais e o Quadro 4 os requisitos não funcionais.

Quadro 3 – Requisitos funcionais

Código	**Descrição**
RF01	Cadastrar, listar, alterar e excluir registros de resíduos (tipo, quantidade, destino, data e fornecedor)
RF02	Filtrar os resíduos por tipo
RF03	Cadastrar, listar e excluir o consumo mensal de água (m³) e energia (kWh)
RF04	Impedir o cadastro de dois consumos do mesmo recurso no mesmo mês
RF05	Cadastrar, listar e excluir fornecedores, informando se possuem ISO 14001
RF06	Impedir a exclusão de fornecedor que tenha resíduos vinculados
RF07	Gerar relatório resumo com total de resíduos, total por tipo e por destino
RF08	Calcular o percentual de resíduos reaproveitados (reciclagem, compostagem e reuso)
RF09	Calcular a variação percentual do consumo de água e energia
RF10	Exportar a lista de resíduos em arquivo CSV
RF11	Exibir um painel com os indicadores e um gráfico de resíduos por tipo

Fonte: Autores, 2026.

Quadro 4 – Requisitos não funcionais

Código	Descrição
RNF01	As funcionalidades devem ser oferecidas por um web service no estilo REST
RNF02	A troca de dados entre cliente e serviço deve ser feita em JSON sobre HTTP
RNF03	O serviço deve validar os dados e devolver mensagens de erro claras com o código HTTP correto
RNF04	O cliente deve funcionar em navegadores atuais sem instalação de programas
RNF05	O sistema deve usar tecnologias gratuitas e de código aberto
RNF06	O código-fonte deve ser versionado em um repositório Git
RNF07	O sistema deve ser simples de instalar e executar em um computador comum

Fonte: Autores, 2026.

4.2.1 Regras de negócio

Além dos requisitos, o grupo definiu algumas regras de negócio, que são as regras que o sistema deve seguir para que os dados façam sentido do ponto de vista da gestão ambiental:

a) a quantidade de resíduo e de consumo deve ser sempre maior que zero;

b) o tipo de resíduo e o destino devem ser escolhidos de uma lista fixa, para que os relatórios possam agrupar os dados corretamente;

c) são considerados reaproveitados os resíduos enviados para reciclagem, compostagem ou reuso; os enviados para aterro ou incineração não entram nesse cálculo, seguindo a ordem de prioridade da Política Nacional de Resíduos Sólidos;

d) cada recurso (água ou energia) só pode ter um registro de consumo por mês;

e) um fornecedor não pode ser excluído se houver resíduos ligados a ele, para não perder o histórico de quem fez a coleta;

f) o vínculo do resíduo com um fornecedor é opcional, pois alguns resíduos, como os orgânicos enviados para compostagem própria, não passam por empresas terceirizadas.

4.3 Ferramentas e tecnologias utilizadas

A escolha das ferramentas levou em conta o conhecimento que o grupo já tinha das disciplinas do curso, a facilidade de aprendizado e o fato de serem gratuitas. O Quadro 5 mostra as ferramentas utilizadas e a função de cada uma no projeto.

Quadro 5 – Ferramentas e tecnologias utilizadas

Ferramenta	**Função no projeto**
Python 3.12	Linguagem usada para implementar o web service
Flask 3	Microframework web usado para criar as rotas da API REST
SQLite	Banco de dados relacional que fica em um único arquivo
HTML5 e CSS3	Estrutura e aparência da aplicação cliente
JavaScript (Fetch API)	Chamadas ao web service a partir do navegador
unittest	Biblioteca do Python usada nos testes automatizados
Playwright	Biblioteca usada nos testes da interface no navegador
Visual Studio Code	Editor de código
Git e GitHub	Controle de versão e repositório do código
Navegador e cURL	Testes manuais das rotas do web service

Fonte: Autores, 2026.

O Python foi escolhido por ser uma linguagem de sintaxe simples, muito usada no mercado e estudada pelo grupo durante o curso. Ele possui uma biblioteca padrão bem completa, que já inclui o módulo sqlite3 para acesso ao banco de dados, o módulo csv para gerar o relatório e o unittest para os testes (Python Software Foundation, 2024).

O Flask é um microframework web para Python. Ele é chamado de micro porque traz apenas o básico, como o roteamento das URLs e o tratamento das requisições e respostas, deixando o restante para o desenvolvedor escolher (Pallets, 2024). Para criar uma rota basta escrever uma função e colocar o decorador @app.route com a URL e os métodos HTTP aceitos. A função jsonify converte dicionários do Python para JSON. Segundo Grinberg (2019), essa simplicidade faz do Flask uma ótima opção para APIs e para quem está aprendendo desenvolvimento web.

O SQLite é um banco de dados relacional que não precisa de um servidor separado, pois todos os dados ficam em um único arquivo no disco (SQLite, 2024). Isso facilitou muito a instalação do sistema nos computadores do grupo e na apresentação, e mesmo assim permitiu usar a linguagem SQL que foi estudada na disciplina de banco de dados. Caso o sistema fosse usado em produção, ele poderia ser trocado por outro banco, como o PostgreSQL, com poucas alterações.

Para a aplicação cliente o grupo optou por HTML, CSS e JavaScript puro, sem frameworks como React ou Angular, para manter o projeto simples e deixar visível como o cliente consome o serviço. As requisições são feitas com a Fetch API, que é a forma nativa dos navegadores atuais de fazer chamadas HTTP de forma assíncrona (Mozilla, 2024).

Por fim, o Git foi utilizado para o controle de versão e o código foi publicado no GitHub. O Git registra o histórico de alterações dos arquivos e permite que várias pessoas trabalhem no mesmo projeto sem perder o trabalho umas das outras (Chacon; Straub, 2014).

4.4 Metodologia de desenvolvimento

O grupo utilizou uma abordagem incremental, que segundo Sommerville (2011) consiste em desenvolver o sistema em partes, entregando e testando uma funcionalidade de cada vez. Primeiro foi construída a estrutura do banco de dados e o módulo de resíduos do web service, testado pelo navegador e pelo cURL. Depois foram acrescentados os módulos de consumo e de fornecedores, em seguida os relatórios e, por último, a aplicação cliente. A cada etapa o código era enviado para o repositório Git, e no final foram escritos os testes automatizados e feitas as correções encontradas.

As etapas planejadas foram as seguintes:

a) estudo do tema: pesquisa sobre web services e sobre a norma ISO 14001;

b) análise: definição do escopo e levantamento dos requisitos;

c) projeto: definição da arquitetura, do modelo do banco de dados e dos endpoints da API;

d) implementação do web service: banco de dados, rotas, validações e relatórios;

e) implementação do cliente: telas, formulários e chamadas ao serviço;

f) testes: testes manuais e testes automatizados com unittest;

g) documentação: redação do trabalho escrito seguindo as normas da ABNT.

4.5 Cronograma

O Quadro 6 apresenta o cronograma seguido pelo grupo durante o semestre. As atividades realizadas também foram registradas na ficha de Atividades Práticas Supervisionadas, que se encontra no Apêndice A.

Quadro 6 – Cronograma de desenvolvimento

Período	Atividade
10/08 a 28/08	Formação do grupo, escolha do tema, pesquisa bibliográfica e proposta ao coordenador
02/09 a 09/09	Levantamento de requisitos e preparação do ambiente de desenvolvimento
16/09	Modelagem do banco de dados e da arquitetura do sistema
23/09 a 07/10	Implementação do web service (resíduos, consumo, fornecedores e relatórios)
14/10 a 21/10	Implementação do cliente web, validações e tratamento de erros
26/10	Testes automatizados e correções
30/10 a 08/11	Redação do trabalho, formatação ABNT e preparação da apresentação

Fonte: Autores, 2026.

4.6 Divisão de tarefas

Apesar de os dois integrantes terem participado das decisões e revisado o trabalho, as tarefas principais foram divididas para que o grupo pudesse avançar ao mesmo tempo em partes diferentes. Um integrante ficou responsável pelo web service, pelo banco de dados e pelos testes, e o outro pela aplicação cliente, pela pesquisa teórica e pela formatação do trabalho escrito. Todos usaram o mesmo repositório Git, o que facilitou juntar as partes. As reuniões aconteceram semanalmente, de forma presencial na universidade ou online, para acompanhar o andamento e resolver dúvidas.

4.7 Recursos necessários

Como todas as ferramentas escolhidas são gratuitas, o projeto não teve custo com licenças. Para o desenvolvimento foram usados os computadores pessoais dos integrantes, com sistema operacional Windows, e os laboratórios da universidade. Para executar o sistema basta um computador com o Python 3 instalado e um navegador atualizado, como o Google Chrome, o Microsoft Edge ou o Mozilla Firefox. Não é necessário instalar um servidor de banco de dados, pois o SQLite já vem junto com o Python, e a única biblioteca externa necessária é o Flask, instalada com o gerenciador de pacotes pip a partir do arquivo requirements.txt.

Para a apresentação do trabalho, o sistema será executado localmente em um computador, com o servidor e o cliente na mesma máquina. Caso fosse colocado em produção, o web service poderia ser hospedado em um servidor na nuvem, com o uso de HTTPS, e o cliente poderia ser acessado por qualquer computador da empresa.

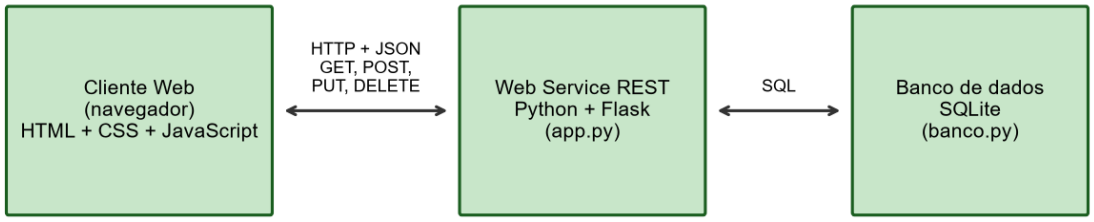
5 PROJETO (ESTRUTURA E MÓDULOS) DO PROGRAMA

Nesta seção é descrita a estrutura do EcoGestão: a arquitetura geral, a organização dos arquivos, o modelo do banco de dados, os módulos e os endpoints oferecidos pelo web service.

5.1 Arquitetura do sistema

O sistema segue a arquitetura cliente-servidor em três camadas, mostrada na Figura 1. A camada de apresentação é a aplicação cliente, que roda no navegador. A camada de serviço é o web service REST feito em Flask, que recebe as requisições, aplica as regras e validações e devolve as respostas em JSON. A camada de dados é o banco SQLite, acessado apenas pelo serviço.

Figura 1 – Arquitetura do sistema EcoGestão



Fonte: Autores, 2026.

Com essa separação, o cliente nunca acessa o banco de dados diretamente. Isso significa que, se no futuro for criado um aplicativo de celular ou se outro sistema da empresa precisar dos dados ambientais, basta consumir o mesmo web service.

5.2 Estrutura de pastas

O projeto foi organizado em duas pastas principais, uma para o servidor e outra para o cliente, como mostrado a seguir:

```
APS---8-Semestre/
|-- servidor/
|   |-- app.py          (web service: rotas da API REST)
|   |-- banco.py        (criacao e acesso ao banco SQLite)
|   |-- test_api.py     (testes do web service)
|   |-- test_interface.py (testes no navegador)
|-- cliente/
|   |-- index.html      (telas da aplicacao)
```

```
|   |-- app.js          (chamadas ao web service)
|   |-- style.css       (aparencia)
|-- documentacao/       (trabalho escrito)
|-- requirements.txt    (dependencias do Python)
|-- README.md           (instrucoes de execucao)
```

5.3 Modelo de dados

O banco de dados possui três tabelas, apresentadas na Figura 2. A tabela fornecedores guarda as partes interessadas, com o campo certificado_iso14001 indicando se o fornecedor possui a certificação. A tabela residuos guarda cada registro de geração de resíduo e possui uma chave estrangeira opcional para o fornecedor que fez a coleta, formando um relacionamento de um para muitos. A tabela consumos guarda o consumo mensal de cada recurso.

Figura 2 – Modelo do banco de dados



Fonte: Autores, 2026.

As tabelas são criadas automaticamente pelo arquivo banco.py na primeira execução do sistema, e alguns dados de exemplo são inseridos para facilitar a demonstração.

5.4 Módulos do sistema

O sistema foi dividido em quatro módulos, que aparecem tanto no web service quanto no cliente.

5.4.1 Módulo de resíduos

É o módulo principal. Permite o cadastro completo (CRUD) dos resíduos gerados. Os tipos aceitos são papel, plástico, vidro, metal, orgânico, eletrônico e rejeito, e os destinos são reciclagem, compostagem, aterro, incineração e reuso. O

serviço confere se o tipo e o destino são válidos, se a quantidade é um número maior que zero e se a data está no formato correto. Também é possível filtrar a listagem por tipo de resíduo.

5.4.2 Módulo de consumo de água e energia

Permite registrar o consumo mensal de água, em metros cúbicos, e de energia, em quilowatt-hora. Como cada recurso só pode ter um registro por mês, o serviço recusa registros duplicados devolvendo o código 409 (conflito).

5.4.3 Módulo de fornecedores

Mantém o cadastro das partes interessadas que prestam serviços ambientais para a empresa. Para manter a integridade dos dados, o serviço não permite excluir um fornecedor que esteja vinculado a algum resíduo.

5.4.4 Módulo de relatórios

Reúne as informações dos outros módulos e calcula os indicadores ambientais: total de resíduos gerados, total por tipo e por destino, percentual de resíduos reaproveitados, variação percentual do consumo de água e energia entre o primeiro e o último mês registrado e quantidade de fornecedores certificados. Esse módulo também gera o arquivo CSV com todos os resíduos, que pode ser aberto em programas de planilha. O percentual reaproveitado é calculado pela fórmula (1):

$$PR = (RC + CP + RU) / T \times 100 \quad (1)$$

Onde PR é o percentual reaproveitado, RC, CP e RU são as quantidades, em quilogramas, enviadas para reciclagem, compostagem e reuso, e T é o total de resíduos gerados.

5.5 Endpoints do web service

O Quadro 7 apresenta todos os endpoints oferecidos pelo web service. Todos ficam abaixo do caminho /api e trocam dados no formato JSON, com exceção da exportação, que devolve um arquivo CSV.

Quadro 7 – Endpoints do web service

Método	URI	Descrição
GET	/api/residuos	Lista os resíduos (filtro opcional ?tipo=)
GET	/api/residuos/{id}	Busca um resíduo
POST	/api/residuos	Cadastra um resíduo
PUT	/api/residuos/{id}	Altera um resíduo
DELETE	/api/residuos/{id}	Exclui um resíduo
GET	/api/consumos	Lista os consumos
POST	/api/consumos	Cadastra um consumo
DELETE	/api/consumos/{id}	Exclui um consumo
GET	/api/fornecedores	Lista os fornecedores
POST	/api/fornecedores	Cadastra um fornecedor
DELETE	/api/fornecedores/{id}	Exclui um fornecedor
GET	/api/relatorios/resumo	Indicadores ambientais
GET	/api/relatorios/residuos.csv	Exporta os resíduos em CSV

Fonte: Autores, 2026.

5.6 Tratamento de erros

Todas as respostas de erro do serviço seguem o mesmo padrão: um objeto JSON com o campo erro contendo a mensagem, acompanhado do código HTTP adequado, como mostrado no Quadro 8. No cliente, a função que chama o serviço verifica o código da resposta e mostra a mensagem de erro em uma faixa vermelha no topo da tela. Se o servidor estiver desligado, o cliente avisa que não foi possível conectar.

Quadro 8 – Códigos HTTP utilizados pelo web service

Código	Significado	Quando ocorre no sistema
200	OK	Consulta, alteração ou exclusão feita com sucesso
201	Created	Registro cadastrado com sucesso
400	Bad Request	Dados inválidos ou faltando
404	Not Found	Registro ou rota não encontrados
409	Conflict	Consumo duplicado no mês ou fornecedor com resíduos vinculados
500	Internal Server Error	Erro inesperado no servidor

Fonte: Autores, 2026.

5.7 Testes

Além dos testes manuais feitos pelo navegador e pelo cURL, foram criados dois conjuntos de testes automatizados, ambos com a biblioteca unittest. Eles usam um banco de dados separado, criado novamente antes de cada teste, para que os dados de exemplo não sejam alterados.

O primeiro conjunto, no arquivo test_api.py, testa o web service diretamente com o cliente de testes do Flask. São 13 testes que verificam a listagem e o filtro por tipo, o cadastro, a alteração e a exclusão de resíduos, a recusa de dados inválidos

(tipo, destino, quantidade, data, fornecedor inexistente e corpo que não é JSON), a busca de registro inexistente, o bloqueio de consumo duplicado e da exclusão de fornecedor com resíduos, a exportação CSV, as rotas inexistentes e os valores calculados no relatório.

O segundo conjunto, no arquivo test_interface.py, faz testes de ponta a ponta: com a biblioteca Playwright, o teste abre o cliente em um navegador de verdade (Microsoft Edge ou Google Chrome) e usa o sistema como um usuário faria, preenchendo formulários e clicando nos botões. São 7 testes que conferem os indicadores do painel, o download do CSV, o ciclo completo de cadastro, filtro, edição e exclusão de um resíduo, a exibição das mensagens de erro, o consumo duplicado, o cadastro e a exclusão de fornecedores e o tratamento de nomes com apóstrofo ou com símbolos de HTML. Esse último teste encontrou um erro real: um nome como D'Agua quebrava o botão Editar e textos com tags HTML eram interpretados pela página. O erro foi corrigido com a função escapar do cliente. Na versão final, os 20 testes passaram.

6 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA

A seguir são apresentados os principais arquivos do código-fonte do sistema. Os arquivos index.html, style.css, test_api.py e test_interface.py não foram incluídos para respeitar o limite de páginas, mas estão disponíveis, junto com todo o projeto, no repositório https://github.com/Caiohere/APS---8-Semestre.

6.1 Arquivo servidor/banco.py

```
# banco.py - acesso ao banco de dados SQLite do EcoGestao
import sqlite3
import os

CAMINHO_BANCO = os.path.join(os.path.dirname(__file__), "ecogestao.db")



def conectar():
    conexao = sqlite3.connect(CAMINHO_BANCO)
    conexao.row_factory = sqlite3.Row  # permite acessar as colunas pelo nome
    return conexao



def criar_tabelas():
    conexao = conectar()
    cursor = conexao.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS fornecedores (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nome TEXT NOT NULL,
            cnpj TEXT NOT NULL,
            tipo TEXT NOT NULL,
            certificado_iso14001 INTEGER NOT NULL DEFAULT 0
        )""")

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS residuos (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            tipo TEXT NOT NULL,
            quantidade_kg REAL NOT NULL,
            destino TEXT NOT NULL,
            data TEXT NOT NULL,
            fornecedor_id INTEGER,
            FOREIGN KEY (fornecedor_id) REFERENCES fornecedores (id)
        )""")

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS consumos (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            recurso TEXT NOT NULL,
            quantidade REAL NOT NULL,
            mes TEXT NOT NULL
        )""")

    conexao.commit()

    # se o banco estiver vazio coloca alguns dados de exemplo
    total = cursor.execute("SELECT COUNT(*) FROM fornecedores").fetchone()[0]
    if total == 0:
        inserir_dados_exemplo(cursor)
        conexao.commit()
```

```
    conexao.close()


def inserir_dados_exemplo(cursor):
    fornecedores = [
        ("Recicla Mais Ltda", "12.345.678/0001-90", "Coleta de reciclaveis", 1),
        ("Verde Transportes", "98.765.432/0001-10", "Transporte de residuos", 0),
        ("EcoAterro SA", "11.222.333/0001-44", "Aterro sanitario", 1),
    ]
    cursor.executemany(
        "INSERT INTO fornecedores (nome, cnpj, tipo, certificado_iso14001) VALUES
(?, ?, ?, ?)",
        fornecedores)

    residuos = [
        ("Papel", 120.5, "Reciclagem", "2026-09-05", 1),
        ("Plastico", 80.0, "Reciclagem", "2026-09-12", 1),
        ("Organico", 200.0, "Compostagem", "2026-09-20", None),
        ("Eletronico", 15.0, "Reciclagem", "2026-10-02", 1),
        ("Rejeito", 95.0, "Aterro", "2026-10-10", 3),
        ("Papel", 100.0, "Reciclagem", "2026-10-18", 1),
    ]
    cursor.executemany(
        "INSERT INTO residuos (tipo, quantidade_kg, destino, data, fornecedor_id) VALUES
(?, ?, ?, ?, ?)",
        residuos)

    consumos = [
        ("Agua", 35.0, "2026-08"), ("Energia", 4200.0, "2026-08"),
        ("Agua", 32.5, "2026-09"), ("Energia", 3900.0, "2026-09"),
        ("Agua", 30.0, "2026-10"), ("Energia", 3650.0, "2026-10"),
    ]
    cursor.executemany(
        "INSERT INTO consumos (recurso, quantidade, mes) VALUES (?, ?, ?)", consumos)
```

6.2 Arquivo servidor/app.py

```
# app.py - Web Service REST do sistema EcoGestao (APS 7/8 semestre)
# Tema: gestao ambiental de uma organizacao seguindo a ISO 14001
import csv
import io
from datetime import datetime

from flask import Flask, jsonify, request, Response, send_from_directory

import banco

app = Flask(__name__, static_folder="../cliente", static_url_path="")

TIPOS_RESIDUO = ["Papel", "Plastico", "Vidro", "Metal", "Organico", "Eletronico", "Rejeito"]
DESTINOS = ["Reciclagem", "Compostagem", "Aterro", "Incineracao", "Reuso"]
RECURSOS = ["Agua", "Energia"]


# ---------- funcoes auxiliares ----------

def erro(mensagem, status=400):
    return jsonify({"erro": mensagem}), status


def data_valida(texto, formato):
    try:
        datetime.strptime(texto, formato)
        return True
```

```
    except (ValueError, TypeError):
        return False


def numero_positivo(valor):
    try:
        return float(valor) > 0
    except (ValueError, TypeError):
        return False


@app.after_request
def liberar_cors(resposta):
    # permite que um cliente em outro endereco consuma o web service
    resposta.headers["Access-Control-Allow-Origin"] = "*"
    resposta.headers["Access-Control-Allow-Headers"] = "Content-Type"
    resposta.headers["Access-Control-Allow-Methods"] = "GET, POST, PUT, DELETE"
    return resposta


@app.errorhandler(404)
def nao_encontrado(e):
    return erro("Recurso nao encontrado", 404)


@app.errorhandler(500)
def erro_interno(e):
    return erro("Erro interno no servidor", 500)


@app.route("/")
def inicio():
    return send_from_directory(app.static_folder, "index.html")


# ---------- fornecedores (stakeholders) ----------

@app.route("/api/fornecedores", methods=["GET"])
def listar_fornecedores():
    con = banco.conectar()
    linhas = con.execute("SELECT * FROM fornecedores ORDER BY nome").fetchall()
    con.close()
    return jsonify([dict(l) for l in linhas])


@app.route("/api/fornecedores", methods=["POST"])
def cadastrar_fornecedor():
    dados = request.get_json(silent=True)
    if not dados:
        return erro("Envie os dados em formato JSON")
    if not dados.get("nome") or not dados.get("cnpj") or not dados.get("tipo"):
        return erro("Os campos nome, cnpj e tipo sao obrigatorios")

    certificado = 1 if dados.get("certificado_iso14001") else 0
    con = banco.conectar()
    cur = con.execute(
        "INSERT INTO fornecedores (nome, cnpj, tipo, certificado_iso14001) VALUES
(?, ?, ?, ?)",
        (dados["nome"], dados["cnpj"], dados["tipo"], certificado))
    con.commit()
    novo_id = cur.lastrowid
    con.close()
    return jsonify({"id": novo_id, "mensagem": "Fornecedor cadastrado"}), 201


@app.route("/api/fornecedores/<int:id>", methods=["DELETE"])
def excluir_fornecedor(id):
```

```
    con = banco.conectar()
    usado = con.execute("SELECT COUNT(*) FROM residuos WHERE fornecedor_id = ?",
(id,)).fetchone()[0]
    if usado > 0:
        con.close()
        return erro("Fornecedor possui residuos vinculados e nao pode ser excluido", 409)
    cur = con.execute("DELETE FROM fornecedores WHERE id = ?", (id,))
    con.commit()
    con.close()
    if cur.rowcount == 0:
        return erro("Fornecedor nao encontrado", 404)
    return jsonify({"mensagem": "Fornecedor excluido"})



# ---------- residuos ----------

def validar_residuo(dados):
    if not dados:
        return "Envie os dados em formato JSON"
    if dados.get("tipo") not in TIPOS_RESIDUO:
        return "Tipo de residuo invalido. Use: " + ", ".join(TIPOS_RESIDUO)
    if dados.get("destino") not in DESTINOS:
        return "Destino invalido. Use: " + ", ".join(DESTINOS)
    if not numero_positivo(dados.get("quantidade_kg")):
        return "A quantidade deve ser um numero maior que zero"
    if not data_valida(dados.get("data"), "%Y-%m-%d"):
        return "Data invalida, use o formato AAAA-MM-DD"
    if dados.get("fornecedor_id"):
        con = banco.conectar()
        existe = con.execute("SELECT id FROM fornecedores WHERE id = ?",
(dados["fornecedor_id"],)).fetchone()
        con.close()
        if existe is None:
            return "Fornecedor informado nao existe"
    return None



@app.route("/api/residuos", methods=["GET"])
def listar_residuos():
    sql = """SELECT r.*, f.nome AS fornecedor FROM residuos r
             LEFT JOIN fornecedores f ON f.id = r.fornecedor_id"""
    parametros = []
    tipo = request.args.get("tipo")  # filtro opcional ?tipo=Papel
    if tipo:
        sql += " WHERE r.tipo = ?"
        parametros.append(tipo)
    sql += " ORDER BY r.data DESC"

    con = banco.conectar()
    linhas = con.execute(sql, parametros).fetchall()
    con.close()
    return jsonify([dict(l) for l in linhas])



@app.route("/api/residuos/<int:id>", methods=["GET"])
def buscar_residuo(id):
    con = banco.conectar()
    linha = con.execute("SELECT * FROM residuos WHERE id = ?", (id,)).fetchone()
    con.close()
    if linha is None:
        return erro("Residuo nao encontrado", 404)
    return jsonify(dict(linha))



@app.route("/api/residuos", methods=["POST"])
def cadastrar_residuo():
    dados = request.get_json(silent=True)
    problema = validar_residuo(dados)
    if problema:
        return erro(problema)
```

```
    con = banco.conectar()
    cur = con.execute(
        "INSERT INTO residuos (tipo, quantidade_kg, destino, data, fornecedor_id) VALUES
(?, ?, ?, ?, ?)",
        (dados["tipo"], float(dados["quantidade_kg"]), dados["destino"], dados["data"],
         dados.get("fornecedor_id") or None))
    con.commit()
    novo_id = cur.lastrowid
    con.close()
    return jsonify({"id": novo_id, "mensagem": "Residuo cadastrado"}), 201



@app.route("/api/residuos/<int:id>", methods=["PUT"])
def alterar_residuo(id):
    dados = request.get_json(silent=True)
    problema = validar_residuo(dados)
    if problema:
        return erro(problema)


    con = banco.conectar()
    cur = con.execute(
        "UPDATE residuos SET tipo = ?, quantidade_kg = ?, destino = ?, data = ?, fornecedor_id
= ? WHERE id = ?",
        (dados["tipo"], float(dados["quantidade_kg"]), dados["destino"], dados["data"],
         dados.get("fornecedor_id") or None, id))
    con.commit()
    con.close()
    if cur.rowcount == 0:
        return erro("Residuo nao encontrado", 404)
    return jsonify({"mensagem": "Residuo alterado"})



@app.route("/api/residuos/<int:id>", methods=["DELETE"])
def excluir_residuo(id):
    con = banco.conectar()
    cur = con.execute("DELETE FROM residuos WHERE id = ?", (id,))
    con.commit()
    con.close()
    if cur.rowcount == 0:
        return erro("Residuo nao encontrado", 404)
    return jsonify({"mensagem": "Residuo excluido"})



# ---------- consumo de agua e energia ----------

@app.route("/api/consumos", methods=["GET"])
def listar_consumos():
    con = banco.conectar()
    linhas = con.execute("SELECT * FROM consumos ORDER BY mes DESC, recurso").fetchall()
    con.close()
    return jsonify([dict(l) for l in linhas])



@app.route("/api/consumos", methods=["POST"])
def cadastrar_consumo():
    dados = request.get_json(silent=True)
    if not dados:
        return erro("Envie os dados em formato JSON")
    if dados.get("recurso") not in RECURSOS:
        return erro("Recurso invalido. Use: Agua ou Energia")
    if not numero_positivo(dados.get("quantidade")):
        return erro("A quantidade deve ser um numero maior que zero")
    if not data_valida(dados.get("mes"), "%Y-%m"):
        return erro("Mes invalido, use o formato AAAA-MM")


    con = banco.conectar()
    existe = con.execute("SELECT id FROM consumos WHERE recurso = ? AND mes = ?",
                         (dados["recurso"], dados["mes"])).fetchone()
    if existe:
        con.close()
        return erro("Ja existe consumo de " + dados["recurso"] + " para este mes", 409)
```

```
    cur = con.execute("INSERT INTO consumos (recurso, quantidade, mes) VALUES (?, ?, ?)",
                      (dados["recurso"], float(dados["quantidade"]), dados["mes"]))
    con.commit()
    novo_id = cur.lastrowid
    con.close()
    return jsonify({"id": novo_id, "mensagem": "Consumo cadastrado"}), 201



@app.route("/api/consumos/<int:id>", methods=["DELETE"])
def excluir_consumo(id):
    con = banco.conectar()
    cur = con.execute("DELETE FROM consumos WHERE id = ?", (id,))
    con.commit()
    con.close()
    if cur.rowcount == 0:
        return erro("Consumo nao encontrado", 404)
    return jsonify({"mensagem": "Consumo excluido"})



# ---------- relatorios ----------

@app.route("/api/relatorios/resumo", methods=["GET"])
def relatorio_resumo():
    con = banco.conectar()
    por_tipo = con.execute(
        "SELECT tipo, SUM(quantidade_kg) AS total FROM residuos GROUP BY tipo ORDER BY total
DESC").fetchall()
    por_destino = con.execute(
        "SELECT destino, SUM(quantidade_kg) AS total FROM residuos GROUP BY
destino").fetchall()
    total = con.execute("SELECT COALESCE(SUM(quantidade_kg), 0) FROM residuos").fetchone()[0]
    consumos = con.execute("SELECT recurso, mes, quantidade FROM consumos ORDER BY
mes").fetchall()
    forn = con.execute(
        "SELECT COUNT(*), COALESCE(SUM(certificado_iso14001), 0) FROM
fornecedores").fetchone()
    con.close()

    # residuos que nao foram para aterro/incineracao sao considerados reaproveitados
    reaproveitado = sum(d["total"] for d in por_destino
                        if d["destino"] in ("Reciclagem", "Compostagem", "Reuso"))
    percentual = round(reaproveitado / total * 100, 1) if total > 0 else 0

    # variacao do consumo entre o primeiro e o ultimo mes cadastrado
    variacao = {}
    for recurso in RECURSOS:
        valores = [c["quantidade"] for c in consumos if c["recurso"] == recurso]
        if len(valores) >= 2:
            variacao[recurso] = round((valores[-1] - valores[0]) / valores[0] * 100, 1)

    return jsonify({
        "total_residuos_kg": total,
        "percentual_reaproveitado": percentual,
        "residuos_por_tipo": [dict(l) for l in por_tipo],
        "residuos_por_destino": [dict(l) for l in por_destino],
        "variacao_consumo_percentual": variacao,
        "fornecedores_total": forn[0],
        "fornecedores_certificados": forn[1],
    })



@app.route("/api/relatorios/residuos.csv", methods=["GET"])
def exportar_csv():
    con = banco.conectar()
    linhas = con.execute("""SELECT r.id, r.data, r.tipo, r.quantidade_kg, r.destino, f.nome
                            FROM residuos r LEFT JOIN fornecedores f ON f.id = r.fornecedor_id
                            ORDER BY r.data""").fetchall()
    con.close()

    saida = io.StringIO()
    escritor = csv.writer(saida, delimiter=";")
```

```
    escritor.writerow(["ID", "Data", "Tipo", "Quantidade (kg)", "Destino", "Fornecedor"])
    for l in linhas:
        escritor.writerow([l[0], l[1], l[2], l[3], l[4], l[5] or ""])
    return Response(saida.getvalue(), mimetype="text/csv",
                    headers={"Content-Disposition": "attachment; filename=residuos.csv"})


if __name__ == "__main__":
    banco.criar_tabelas()
    app.run(host="0.0.0.0", port=5000, debug=True)
```

6.3 Arquivo cliente/app.js

```
// app.js - cliente que consome o web service do EcoGestao
// se a pagina foi aberta direto do arquivo usa o servidor local, senao usa o mesmo endereco
const API = location.protocol === "file:" ? "http://localhost:5000/api" : "/api";
document.getElementById("linkCsv").href = API + "/relatorios/residuos.csv";

// funcao generica para chamar o web service
async function chamarApi(caminho, metodo = "GET", corpo = null) {
    const opcoes = { method: metodo, headers: { "Content-Type": "application/json" } };
    if (corpo) opcoes.body = JSON.stringify(corpo);
    try {
        const resposta = await fetch(API + caminho, opcoes);
        const dados = await resposta.json();
        if (!resposta.ok) {
            mostrarMensagem(dados.erro || "Erro na requisição", true);
            return null;
        }
        return dados;
    } catch (e) {
        mostrarMensagem("Não foi possível conectar ao servidor. Ele está ligado?", true);
        return null;
    }
}

function mostrarMensagem(texto, ehErro = false) {
    const div = document.getElementById("mensagem");
    div.textContent = texto;
    div.className = ehErro ? "erro" : "sucesso";
    setTimeout(() => { div.textContent = ""; div.className = ""; }, 4000);
}

function mostrarTela(id) {
    document.querySelectorAll(".tela").forEach(t => t.style.display = "none");
    document.getElementById(id).style.display = "block";
    if (id === "painel") carregarPainel();
    if (id === "residuos") { carregarFornecedoresSelect(); carregarResiduos(); }
    if (id === "consumos") carregarConsumos();
    if (id === "fornecedores") carregarFornecedores();
}

// evita que textos digitados pelo usuario sejam interpretados como HTML
function escapar(texto) {
    const div = document.createElement("div");
    div.textContent = texto;
    return div.innerHTML;
}

function formatarData(data) {
    const p = data.split("-");
    return p.length === 3 ? `${p[2]}/${p[1]}/${p[0]}` : `${p[1]}/${p[0]}`;
}

// ---------- painel ----------
async function carregarPainel() {
    const r = await chamarApi("/relatorios/resumo");
    if (!r) return;
```

```
    document.getElementById("totalResiduos").textContent = r.total_residuos_kg.toFixed(1);
    document.getElementById("percReaproveitado").textContent = r.percentual_reaproveitado +
"%";
    const v = r.variacao_consumo_percentual;
    document.getElementById("varAgua").textContent = v.Agua !== undefined ? v.Agua + "%" :
"-";
    document.getElementById("varEnergia").textContent = v.Energia !== undefined ? v.Energia +
"%" : "-";
    document.getElementById("fornCertificados").textContent =
        r.fornecedores_certificados + " de " + r.fornecedores_total;

    // grafico de barras simples feito com div
    const maior = Math.max(...r.residuos_por_tipo.map(x => x.total), 1);
    document.getElementById("grafico").innerHTML = r.residuos_por_tipo.map(x => `
        <div class="barra-linha">
            <span class="rotulo">${x.tipo}</span>
            <div class="barra" style="width:${x.total / maior * 70}%"></div>
            <span>${x.total.toFixed(1)}</span>
        </div>`).join("");
}

// ---------- residuos ----------
let residuosCarregados = [];

async function carregarResiduos() {
    const filtro = document.getElementById("filtroTipo").value;
    const lista = await chamarApi("/residuos" + (filtro ? "?tipo=" + filtro : ""));
    if (!lista) return;
    residuosCarregados = lista;
    document.getElementById("tabelaResiduos").innerHTML = lista.map(r => `
        <tr>
            <td>${formatarData(r.data)}</td><td>${r.tipo}</td><td>${r.quantidade_kg}</td>
            <td>${r.destino}</td><td>${escapar(r.fornecedor || "-")}</td>
            <td><button onclick="editarResiduo(${r.id})">Editar</button>
                <button onclick="excluir('/residuos/${r.id}',
carregarResiduos)">Excluir</button></td>
        </tr>`).join("");
}

async function carregarFornecedoresSelect() {
    const lista = await chamarApi("/fornecedores");
    if (!lista) return;
    document.getElementById("fornecedorResiduo").innerHTML = '<option value="">Sem
fornecedor</option>' +
        lista.map(f => `<option value="${f.id}">${escapar(f.nome)}</option>`).join("");
}

function editarResiduo(id) {
    const r = residuosCarregados.find(x => x.id === id);
    document.getElementById("residuoId").value = r.id;
    document.getElementById("tipo").value = r.tipo;
    document.getElementById("quantidade").value = r.quantidade_kg;
    document.getElementById("destino").value = r.destino;
    document.getElementById("data").value = r.data;
    document.getElementById("fornecedorResiduo").value = r.fornecedor_id || "";
}

document.getElementById("formResiduo").addEventListener("submit", async (e) => {
    e.preventDefault();
    const id = document.getElementById("residuoId").value;
    const dados = {
        tipo: document.getElementById("tipo").value,
        quantidade_kg: document.getElementById("quantidade").value,
        destino: document.getElementById("destino").value,
        data: document.getElementById("data").value,
        fornecedor_id: document.getElementById("fornecedorResiduo").value || null
    };
    const r = id ? await chamarApi("/residuos/" + id, "PUT", dados)
                 : await chamarApi("/residuos", "POST", dados);
    if (r) {
        mostrarMensagem(r.mensagem);
        e.target.reset();
        document.getElementById("residuoId").value = "";
        carregarResiduos();
```

```
    }
});

// ---------- consumos ----------
async function carregarConsumos() {
    const lista = await chamarApi("/consumos");
    if (!lista) return;
    document.getElementById("tabelaConsumos").innerHTML = lista.map(c => `
        <tr>
            <td>${formatarData(c.mes)}</td><td>${c.recurso}</td>
            <td>${c.quantidade} ${c.recurso === "Agua" ? "m³" : "kWh"}</td>
            <td><button onclick="excluir('/consumos/${c.id}',
carregarConsumos)">Excluir</button></td>
        </tr>`).join("");
}

document.getElementById("formConsumo").addEventListener("submit", async (e) => {
    e.preventDefault();
    const r = await chamarApi("/consumos", "POST", {
        recurso: document.getElementById("recurso").value,
        quantidade: document.getElementById("qtdConsumo").value,
        mes: document.getElementById("mes").value
    });
    if (r) { mostrarMensagem(r.mensagem); e.target.reset(); carregarConsumos(); }
});

// ---------- fornecedores ----------
async function carregarFornecedores() {
    const lista = await chamarApi("/fornecedores");
    if (!lista) return;
    document.getElementById("tabelaFornecedores").innerHTML = lista.map(f => `
        <tr>
            <td>${escapar(f.nome)}</td><td>${escapar(f.cnpj)}</td><td>${escapar(f.tipo)}</td>
            <td>${f.certificado_iso14001 ? "Sim" : "Não"}</td>
            <td><button onclick="excluir('/fornecedores/${f.id}',
carregarFornecedores)">Excluir</button></td>
        </tr>`).join("");
}

document.getElementById("formFornecedor").addEventListener("submit", async (e) => {
    e.preventDefault();
    const r = await chamarApi("/fornecedores", "POST", {
        nome: document.getElementById("nomeForn").value,
        cnpj: document.getElementById("cnpj").value,
        tipo: document.getElementById("tipoForn").value,
        certificado_iso14001: document.getElementById("certificado").checked
    });
    if (r) { mostrarMensagem(r.mensagem); e.target.reset(); carregarFornecedores(); }
});

// exclusao generica usada pelas tres telas
async function excluir(caminho, recarregar) {
    if (!confirm("Deseja realmente excluir este registro?")) return;
    const r = await chamarApi(caminho, "DELETE");
    if (r) { mostrarMensagem(r.mensagem); recarregar(); }
}

// abre a tela indicada no endereco (ex: index.html#residuos) ou o painel
mostrarTela(location.hash.slice(1) || "painel");
```

7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO

Nesta seção são mostradas as telas do sistema em funcionamento e explicado como executá-lo. Na apresentação para o professor o sistema será executado em um computador, demonstrando todas as funcionalidades descritas.

7.1 Como executar o sistema

Para executar o sistema é preciso ter o Python 3 instalado. Os comandos são:

```
git clone https://github.com/Caiohere/APS---8-Semestre.git
cd APS---8-Semestre
pip install -r requirements.txt
cd servidor
python app.py
```

Depois disso, basta abrir o navegador no endereço http://localhost:5000. Os testes automatizados podem ser executados com os comandos python test_api.py e python test_interface.py dentro da pasta servidor (o segundo precisa do comando pip install playwright).

7.2 Telas do sistema

A Figura 3 mostra o painel de indicadores, que é a tela inicial. Nele aparecem o total de resíduos gerados, o percentual reaproveitado, a variação do consumo de água e energia e a quantidade de fornecedores certificados, além de um gráfico de barras com os resíduos por tipo e o botão para exportar o relatório em CSV.

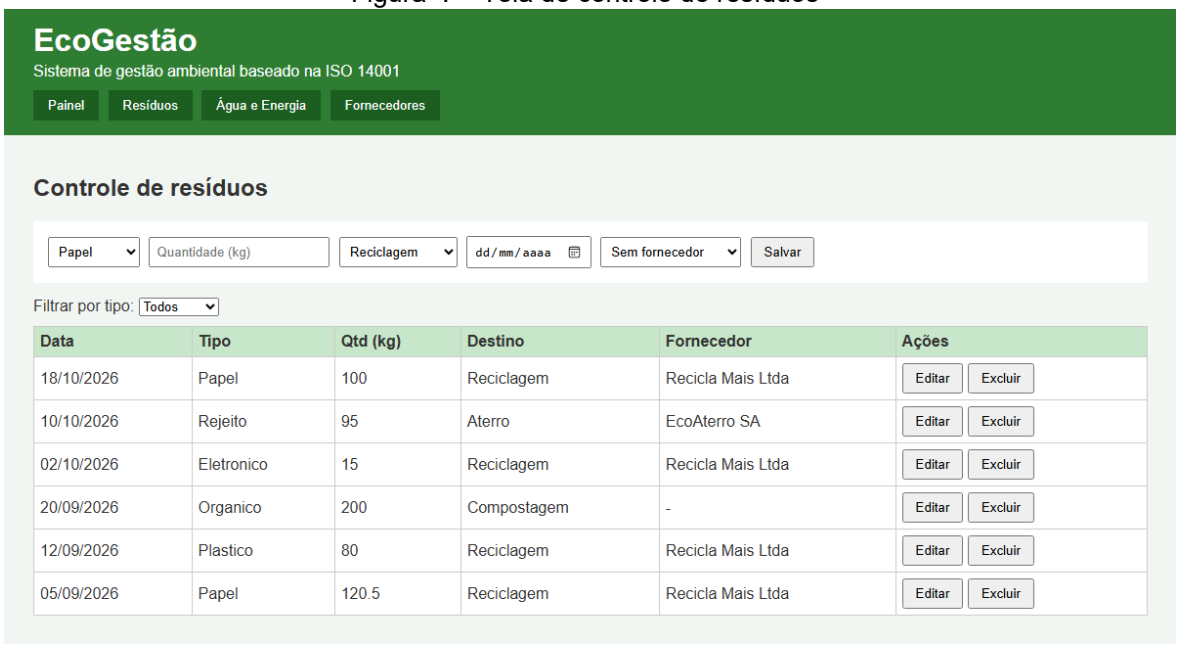
Figura 3 – Painel de indicadores ambientais



Fonte: Autores, 2026.

A Figura 4 mostra a tela de controle de resíduos, com o formulário de cadastro, o filtro por tipo e a tabela com os registros. Ao clicar em Editar, os dados do registro são carregados no formulário, e ao salvar o cliente envia um PUT para o serviço.

Figura 4 – Tela de controle de resíduos



Fonte: Autores, 2026.

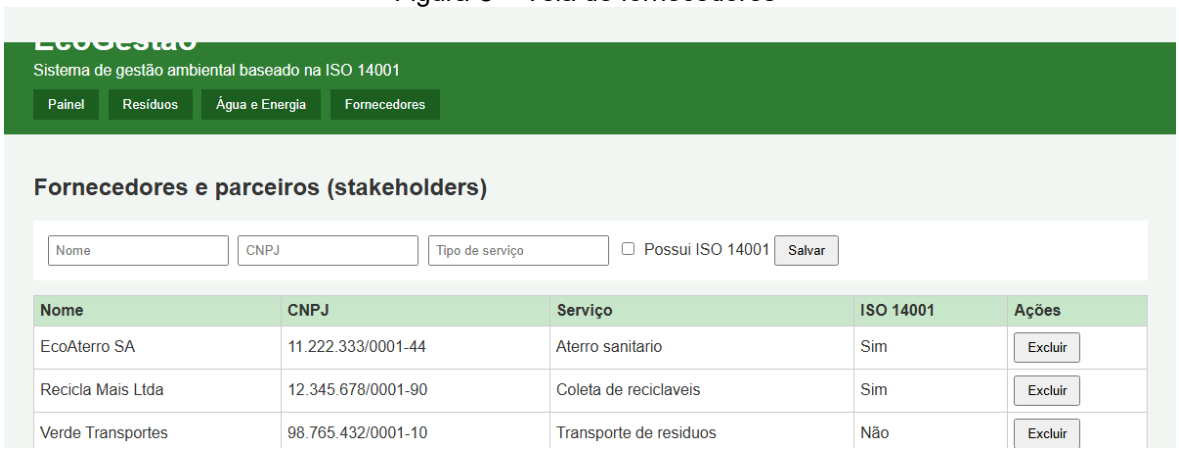
As Figuras 5 e 6 mostram as telas de consumo de água e energia e de fornecedores.

Figura 5 – Tela de consumo de água e energia



Fonte: Autores, 2026.

Figura 6 – Tela de fornecedores



Fonte: Autores, 2026.

Como o sistema é um web service, as rotas também podem ser acessadas diretamente, sem o cliente. A Figura 7 mostra a resposta em JSON da rota /api/residuos?tipo=Papel aberta no navegador, e a Figura 8 mostra a resposta de erro quando é pedido um resíduo que não existe.

Figura 7 – Resposta em JSON do web service

Estilos de formatação ☐


```
[
  {
    "data": "2026-10-18",
    "destino": "Reciclagem",
    "fornecedor": "Recicla Mais Ltda",
    "fornecedor_id": 1,
    "id": 6,
    "quantidade_kg": 100.0,
    "tipo": "Papel"
  },
  {
    "data": "2026-09-05",
    "destino": "Reciclagem",
    "fornecedor": "Recicla Mais Ltda",
    "fornecedor_id": 1,
    "id": 1,
    "quantidade_kg": 120.5,
    "tipo": "Papel"
  }
]
```


Fonte: Autores, 2026.

Figura 8 – Resposta de erro 404 do web service

Estilos de formatação ☐


```
{
  "erro": "Residuo nao encontrado"
}
```


Fonte: Autores, 2026.

8 CONSIDERAÇÕES FINAIS

Este trabalho teve como objetivo desenvolver uma aplicação baseada em web services ligada à gestão ambiental de uma organização, e esse objetivo foi atingido. O EcoGestão oferece um web service REST com os módulos de resíduos, consumo de água e energia, fornecedores e relatórios, e uma aplicação cliente que consome esse serviço. Todos os requisitos levantados foram implementados, incluindo as funções extras de filtro, painel com gráfico, exportação em CSV e testes automatizados do serviço e da interface.

8.1 Elementos utilizados no desenvolvimento

Durante o projeto foram utilizados diversos elementos estudados na seção 3. O protocolo HTTP foi usado em toda a comunicação, com os métodos GET, POST, PUT e DELETE representando as operações de consulta, cadastro, alteração e exclusão. O formato JSON foi usado para representar os dados. O estilo REST orientou o projeto das URIs, organizadas em torno dos recursos resíduos, consumos, fornecedores e relatórios. Os códigos de status do HTTP foram a base do tratamento de erros, e o mecanismo de CORS precisou ser entendido para permitir o acesso do cliente. Do lado do servidor, foram usados a linguagem Python, o Flask e o banco SQLite com consultas SQL parametrizadas; do lado do cliente, HTML, CSS e JavaScript com a Fetch API. O Git e o GitHub foram usados para o controle de versão.

Uma dificuldade encontrada foi entender, no começo, a diferença entre SOAP e REST e qual seria o mais adequado. Depois de estudar os dois, o grupo concluiu que o REST era mais simples e atendia bem ao projeto. Outra dificuldade foi o erro de CORS que apareceu quando o cliente foi aberto direto do arquivo, que foi resolvido acrescentando os cabeçalhos corretos nas respostas do serviço. Como melhorias futuras, o grupo sugere a criação de autenticação de usuários, o cadastro de metas ambientais com alertas quando o consumo passar da meta, mais gráficos e um aplicativo de celular consumindo o mesmo serviço.

8.2 Efeito do trabalho na formação

O trabalho foi importante para a formação do grupo porque permitiu colocar em prática, em um único projeto, muitos conteúdos que normalmente são vistos de forma separada no curso. Foi preciso pensar desde o levantamento dos requisitos até os testes, passando pela modelagem do banco de dados e pela programação do servidor e do cliente, o que se parece muito com o que acontece em um projeto real de uma empresa. O contato com web services, APIs REST e versionamento com Git também é muito valorizado no mercado de trabalho, já que quase todo sistema atual depende dessas tecnologias.

Além da parte técnica, o trabalho em grupo exigiu organização, divisão de tarefas e comunicação, e a redação do documento seguindo as normas da ABNT ajudou a desenvolver a escrita acadêmica. Por fim, o tema ambiental fez o grupo refletir sobre a responsabilidade que os profissionais de computação também têm com a sociedade e o meio ambiente, mostrando que a tecnologia pode ser usada para ajudar as empresas a reduzir os seus impactos.

8.3 Interdisciplinaridade

O projeto envolveu conhecimentos de várias disciplinas do curso. De Engenharia de Software vieram o levantamento de requisitos, a metodologia incremental e os testes. De Banco de Dados vieram a modelagem das tabelas, os relacionamentos e a linguagem SQL. De Redes de Computadores vieram o funcionamento do protocolo HTTP e o modelo cliente-servidor. De Linguagens de Programação e de Desenvolvimento Web vieram o Python, o HTML, o CSS e o JavaScript. De Sistemas Distribuídos vieram os conceitos de serviços, interoperabilidade e SOA. De Segurança da Informação vieram os cuidados com validação de dados e injeção de SQL. E de disciplinas ligadas a gestão, ética e meio ambiente vieram os conceitos de gestão ambiental, partes interessadas, ISO 14001 e a Política Nacional de Resíduos Sólidos, que deram sentido ao sistema.

Essa integração mostrou ao grupo que um sistema de verdade não depende só de saber programar, mas também de entender o problema do usuário e o contexto em que ele está inserido, que neste caso foi a necessidade das empresas de cuidar melhor do meio ambiente.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO 14001**: sistemas de gestão ambiental: requisitos com orientações para uso. 3. ed. Rio de Janeiro: ABNT, 2015.

BARBIERI, José Carlos. **Gestão ambiental empresarial**: conceitos, modelos e instrumentos. 4. ed. São Paulo: Saraiva, 2016.

BIBLIOTECA DA UNIVERSIDADE PAULISTA - UNIP. **Guia de normalização para apresentação de trabalhos acadêmicos da Universidade Paulista**: ABNT. São Paulo: UNIP, 2025. Disponível em: https://unip.br/servicos/biblioteca/guia.aspx. Acesso em: 30 out. 2026.

BRASIL. Lei nº 12.305, de 2 de agosto de 2010. Institui a Política Nacional de Resíduos Sólidos; altera a Lei nº 9.605, de 12 de fevereiro de 1998; e dá outras providências. **Diário Oficial da União**: seção 1, Brasília, DF, 3 ago. 2010. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2010/lei/l12305.htm. Acesso em: 24 ago. 2026.

BRAY, Tim (ed.). **The JavaScript Object Notation (JSON) data interchange format**. RFC 8259. [S. l.]: IETF, 2017. Disponível em: https://www.rfc-editor.org/rfc/rfc8259. Acesso em: 19 ago. 2026.

CHACON, Scott; STRAUB, Ben. **Pro Git**. 2. ed. New York: Apress, 2014. Disponível em: https://git-scm.com/book/pt-br/v2. Acesso em: 9 set. 2026.

ERL, Thomas. **SOA**: princípios de design de serviços. São Paulo: Pearson Prentice Hall, 2009.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Information and Computer Science) – University of California, Irvine, 2000. Disponível em: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm. Acesso em: 19 ago. 2026.

FIELDING, Roy Thomas; NOTTINGHAM, Mark; RESCHKE, Julian (ed.). **HTTP semantics**. RFC 9110. [S. l.]: IETF, 2022. Disponível em: https://www.rfc-editor.org/rfc/rfc9110. Acesso em: 19 ago. 2026.

GRINBERG, Miguel. **Desenvolvimento web com Flask**: desenvolvendo aplicações web com Python. São Paulo: Novatec, 2019.

MOZILLA. **MDN Web Docs**: Fetch API e Cross-Origin Resource Sharing (CORS). [S. l.]: Mozilla, 2024. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API. Acesso em: 14 out. 2026.

PALLETS. **Flask documentation**. [S. l.]: Pallets, 2024. Disponível em: https://flask.palletsprojects.com/. Acesso em: 23 set. 2026.

PYTHON SOFTWARE FOUNDATION. **Python 3 documentation**. [S. l.]: PSF, 2024. Disponível em: https://docs.python.org/3/. Acesso em: 23 set. 2026.

RICHARDSON, Leonard; RUBY, Sam. **RESTful web services**. Sebastopol: O'Reilly, 2007.

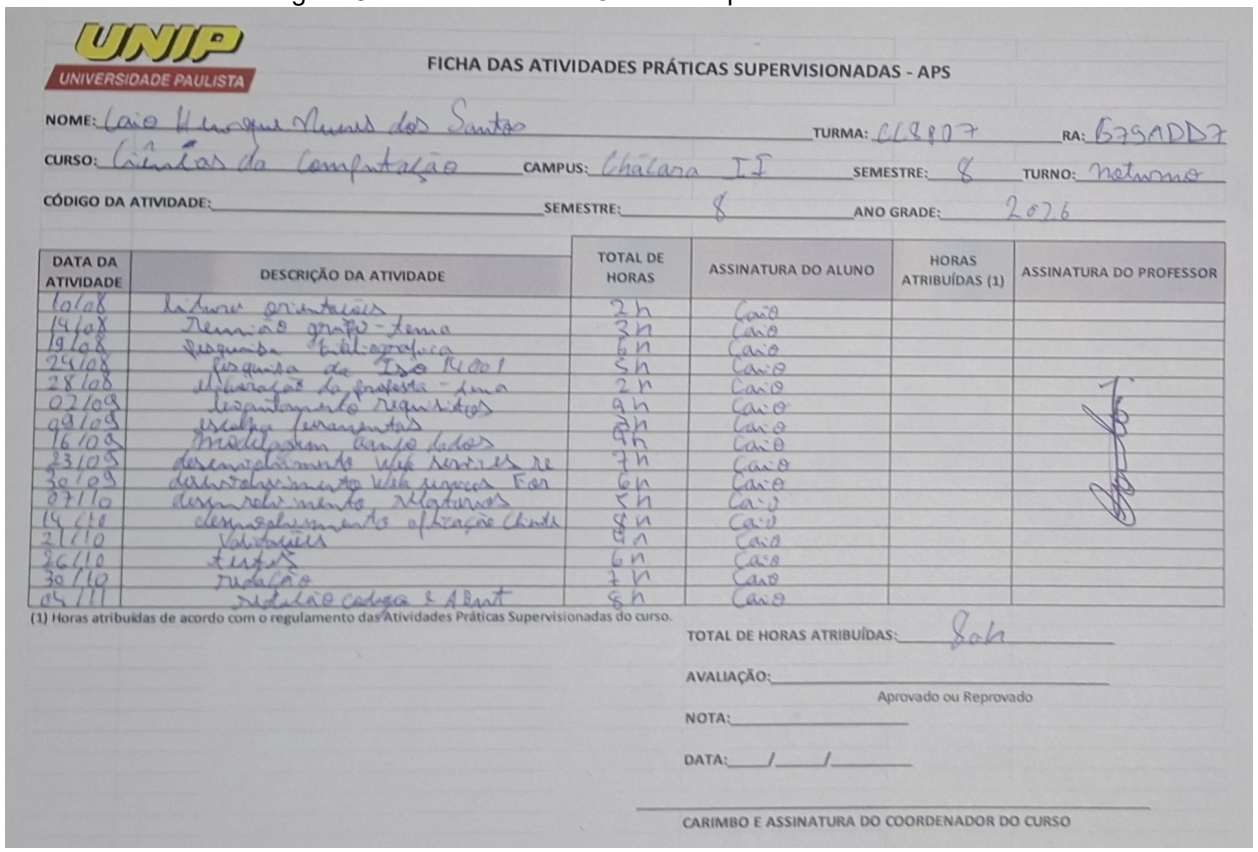
SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SQLITE. **About SQLite**. [S. l.]: SQLite, 2024. Disponível em: https://www.sqlite.org/about.html. Acesso em: 16 set. 2026.

W3C. **Web services architecture**. W3C Working Group Note. [S. l.]: W3C, 2004. Disponível em: https://www.w3.org/TR/ws-arch/. Acesso em: 19 ago. 2026.

APÊNDICE A – FICHAS DE ATIVIDADES PRÁTICAS SUPERVISIONADAS

Figura 9 – Ficha de APS – Caio Henrique Nunes dos Santos



UNIP
UNIVERSIDADE PAULISTA

FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

NOME: Caio Henrique Nunes dos Santos TURMA: CC8P07 RA: B79ADD7

CURSO: Ciências da Computação CAMPUS: Chácara II SEMESTRE: 8 TURNO: noturno

CÓDIGO DA ATIVIDADE: SEMESTRE: 8 ANO GRADE: 2026

DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
7/08	Leituras orientadas	2h	Caio		
14/08	Reunião grupo - tema	3h	Caio		
19/08	pesquisa bibliográfica	6h	Caio		
24/08	pesquisa de IDE Rad	5h	Caio		
28/08	elaboração do projeto - lima	2h	Caio		
02/09	levantamento requisitos	9h	Caio		
09/09	escolha ferramentas	3h	Caio		
16/09	Modelagem dados	4h	Caio		
23/09	desenvolvimento Web services re	7h	Caio		
30/09	desenvolvimento Web pessoa Fis	6h	Caio		
07/10	desenvolvimento relatorios	5h	Caio		
14/10	desenvolvimento aplicação Cliente	8h	Caio		
21/10	Validações	4h	Caio		
26/10	testes	6h	Caio		
30/10	redação	7h	Caio		
04/11	redação codigo e Abstr	8h	Caio		

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS: 80h

AVALIAÇÃO: Aprovado ou Reprovado

NOTA:

DATA: / /

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO

Fonte: Autores, 2026.

Figura 10 – Ficha de APS – Carlos Eduardo Bispo Souza



UNIP
UNIVERSIDADE PAULISTA

FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

NOME: Carlos Eduardo Bispo Souza TURMA: CC8P07 RA: G7260A2

CURSO: Ciências da Computação CAMPUS: Chácara SEMESTRE: 8º TURNO: NOTURNO

CÓDIGO DA ATIVIDADE: 77B5 SEMESTRE: 8º ANO GRADE: 2026

DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
10/08	Leitura das orientações	2H	Carlos		
11/08	Reunião para definição do tema	3h	Carlos		
12/08	Pesquisa bibliográfica	6h	Carlos		
21/08	Pesquisa ISO e política de revisão	3h	Carlos		
28/08	Elaboração do proposta	2h	Carlos		
2/9	Levantamento dos requisito do sistema	4h	Carlos		
8/9	escolha do paradigmento	3h	Carlos		
16/9	modelagem do banco de dados	4h	Carlos		
23/9	Desenvolvimento do web service	7h	Carlos		
30/9	Desenvolvimento dos relatorios	5h	Carlos		
5/10	Desenvolvimento do web consumo	6h	Carlos		
14/10	Desenvolvimento app cliente	8h	Carlos		
21/10	Validação e tratamento de erros	4h	Carlos		
26/10	Testes automatizados e correções	6h	Carlos		
30/10	Redação: objetivos, introdução de	7h	Carlos		
4/11	Redação do projeto e codigo	8h	Carlos		

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS: 80h

AVALIAÇÃO: _____
Aprovado ou Reprovado

NOTA: _____

DATA: ___/___/_____

_____
CARIMBO E ASSINATURA DO COORDENADOR DO CURSO

Fonte: Autores, 2026.